Introducción a la programación orientada a objetos: diseño e implementación de la clase Estudiante en Java

Grupo 1

Dereck Shair Paredes Acosta

Brishy Anahy Ashanga Yumbo

Fher Dorian Calderon Carvajal

Casillas Ochoa Antony Sebastián

Karina Paola Sánchez Bastidas

Carrera de Software, Facultad de Ingeniería en Sistemas, Electrónica e Industrial

Universidad Técnica de Ambato

Algoritmos y Lógica de Programación

Ing. José Caiza

15 de octubre de 2026

Introducción a la programación orientada a objetos: diseño e implementación de la clase Estudiante en Java

La Universidad Técnica de Ambato necesita un sistema académico que permita registrar la información básica de sus estudiantes. Como futuros ingenieros de software, en esta práctica construimos el primer componente de ese sistema: una clase en Java que representa a un estudiante y a partir de la cual se pueden crear tantos registros como se necesiten. Aunque el programa es corto, nos sirvió para entender de forma práctica las ideas en las que se basa la programación orientada a objetos (POO), que es la forma en que se desarrolla la mayor parte del software actual.

El objetivo del trabajo fue comprender y aplicar los conceptos fundamentales de la POO mediante el diseño e implementación de una clase en Java, identificando sus atributos, creando objetos y evidenciando el trabajo colaborativo en GitHub. El documento sigue el orden de las actividades de la guía: primero explicamos los conceptos con nuestras propias palabras, después presentamos el diseño de la clase `Estudiante`, su implementación y ejecución, la organización del repositorio y, al final, la reflexión del grupo.

Análisis y conceptualización

¿Qué es la programación orientada a objetos?

La programación orientada a objetos es una manera de organizar un programa tomando como base las cosas del problema real. En lugar de escribir una lista larga de instrucciones con variables sueltas, se identifican los elementos que intervienen, por ejemplo un estudiante, una materia o una matrícula, y cada uno se representa como un objeto que agrupa sus datos y lo que puede hacer (Oracle, s.f.-a). Para nosotros, la idea central es que el código se parece más a la realidad que queremos representar, y por eso resulta más fácil de leer, de corregir y de ampliar.

Java es un lenguaje pensado para este paradigma: todo el código se escribe dentro de clases y el programa funciona creando objetos y haciendo que trabajen entre sí (Deitel y Deitel, 2016). Por eso es un buen lenguaje para aprender POO desde el principio.

¿Cuál es la diferencia entre una clase y un objeto?

Una clase es el molde o la plantilla, y un objeto es algo concreto que se fabrica con ese molde. La clase describe qué características va a tener cada objeto, pero por sí sola no guarda los datos de nadie. El objeto, en cambio, existe en la memoria mientras el programa se ejecuta y tiene sus propios valores (Joyanes Aguilar y Zahonero Martínez, 2011). En nuestro caso, `Estudiante` es la clase y "Daniela Villacres, 18 años, estudiante de Software de primer semestre" es un objeto de esa clase. Con una sola clase se pueden crear todos los objetos que hagan falta.

¿Qué es un atributo?

Un atributo es una característica o dato que describe a un objeto. Se declara dentro de la clase indicando un tipo de dato y un nombre, por ejemplo `String nombre` o `int edad`. Todos los objetos de una misma clase tienen los mismos atributos, pero cada uno guarda en ellos sus propios valores (Booch et al., 2007). Dicho de otra forma, el atributo es la casilla de un formulario y el valor es lo que se escribe en esa casilla.

¿Qué significa instanciar una clase?

Instanciar es crear un objeto a partir de una clase. En Java se hace con el operador `new`, que reserva un espacio en la memoria para el objeto nuevo y devuelve una referencia a ese espacio, la cual normalmente se guarda en una variable (Oracle, s.f.-b). Por ejemplo, en la instrucción `Estudiante estudiante1 = new Estudiante();` se crea un objeto de tipo `Estudiante` y la variable `estudiante1` queda apuntando a él. Por esa razón, a cada objeto también se le llama instancia de la clase.

¿Qué ventajas tiene representar un estudiante mediante un objeto?

Si quisiéramos guardar los datos de tres estudiantes sin usar objetos, necesitaríamos quince variables independientes: `nombre1`, `apellido1`, `edad1` y así hasta `semestre3`. Con tres personas todavía se puede manejar, pero con cien o mil estudiantes sería casi imposible mantener el orden, y sería muy fácil mezclar el nombre de un estudiante con la edad de otro. Al usar un objeto, todos los datos de una persona quedan juntos dentro de una sola variable, y la estructura se define una sola vez en la clase. En la Tabla 1 resumimos las diferencias que encontramos entre las dos formas de trabajar.

Tabla 1

Comparación entre el uso de variables independientes y el uso de objetos

Criterio	Variables independientes	Objeto Estudiante
Datos de tres estudiantes	15 variables sueltas (`nombre1`, `edad1`, `nombre2`…)	3 objetos, cada uno con sus 5 atributos
Organización	Los datos de una misma persona quedan dispersos	Los datos de cada persona están agrupados
Riesgo de errores	Alto: es fácil mezclar los datos de dos estudiantes	Bajo: cada valor pertenece a un solo objeto
Agregar un dato nuevo (p. ej., el correo)	Hay que crear una variable más por cada estudiante	Se agrega un atributo en la clase y todos los objetos lo tienen
Reutilización	El código sirve solo para ese programa	La clase se puede usar en otros módulos del sistema

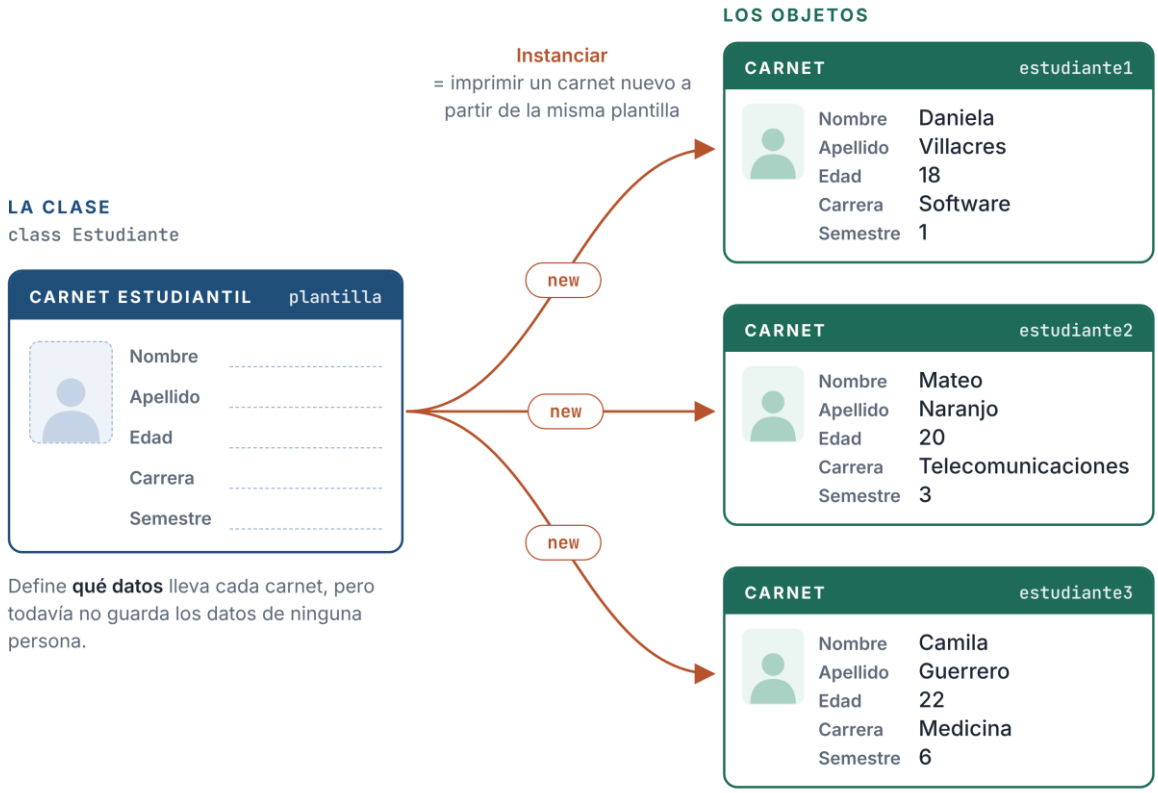
Nota. Para el conteo se consideran tres estudiantes con los cinco datos solicitados en la guía.

Ejemplo de la vida cotidiana: el carnet estudiantil

Para explicar los conceptos usamos el ejemplo del carnet estudiantil. Antes de imprimir los carnets, la universidad diseña una plantilla que indica qué datos debe llevar cada uno: nombre, apellido, edad, carrera y semestre. Esa plantilla es la clase. Cada vez que se imprime el carnet de un estudiante en particular se está creando un objeto, es decir, se está instanciando la clase. Los campos de la plantilla son los atributos, y lo que se escribe en ellos (Daniela, Villacres, 18, Software, 1) son los valores de ese objeto. Con la misma plantilla se pueden imprimir miles de carnets diferentes, como se observa en la Figura 1.

Figura 1

Analogía entre la plantilla del carnet estudiantil y la clase Estudiante



Nota. La plantilla representa la clase; cada carnet impreso representa un objeto con sus propios valores. Los datos de los estudiantes son ficticios.

Diseño de la solución

Para diseñar la clase partimos de una pregunta sencilla: ¿qué datos necesita el sistema académico para registrar a un estudiante? La guía definió cinco atributos: nombre, apellido, edad, carrera y semestre. Para los datos que son texto elegimos el tipo `String`, y para la edad y el semestre usamos `int`, porque son números enteros que no llevan decimales. Como en este tema solo se trabajan clases, atributos y objetos, la clase no incluye métodos ni constructores propios.

Diagrama de la clase Estudiante

La Figura 2 muestra la clase en notación UML (Lenguaje Unificado de Modelado). En UML una clase se dibuja como un rectángulo dividido en tres partes: el nombre, los atributos y los métodos (Fowler, 2004). Cada atributo se escribe con su visibilidad, su nombre y su tipo. Usamos el signo +, que significa público, para poder asignar los valores directamente desde la clase `Principal`; más adelante, cuando se estudie el encapsulamiento, estos atributos deberían pasar a ser privados.

Figura 2

Diagrama UML de la clase Estudiante



Nota. El compartimento de métodos está vacío porque en este tema la clase solo define atributos.

Información que representa cada atributo

En la Tabla 2 se explica qué información representa cada atributo, qué valores puede almacenar y un ejemplo de cada uno, tomado del primer objeto que se crea en el programa.

Tabla 2

Descripción de los atributos de la clase Estudiante

Atributo	Tipo	Información que representa	Valores que puede almacenar	Ejemplo
`nombre`	`String`	Nombre o nombres del estudiante	Texto con letras y espacios	`"Daniela"`
`apellido`	`String`	Apellido o apellidos del estudiante	Texto con letras y espacios	`"Villacres"`
`edad`	`int`	Edad en años cumplidos	Número entero mayor que cero	`18`
`carrera`	`String`	Carrera en la que está matriculado	Nombre de una carrera de la universidad	`"Software"`
`semestre`	`int`	Semestre o nivel que cursa	Número entero, por ejemplo del 1 al 10	`1`

Nota. Los rangos indicados son referenciales; en esta versión el programa todavía no valida los valores que se asignan.

Implementación en Java

Creación del proyecto

Una vez definido el diseño pasamos al código. Creamos un proyecto Java con Maven, que es el tipo de proyecto que Apache NetBeans genera por defecto y que Visual Studio Code también reconoce con la extensión *Extension Pack for Java*; así cualquier integrante puede abrirlo en el entorno que prefiera. Las dos clases se ubicaron en el paquete `ec.edu.uta.sistemaacademico`, cuyo nombre sigue la convención de usar el dominio de la institución escrito al revés. La Figura 3 muestra la estructura final del proyecto.

Figura 3

Estructura del proyecto POO-Tema1-Grupo1

```
POO-Tema1-Grupo1/
├── docs/                 ← diagramas de clase y de objetos
├── evidencias/           ← capturas de la ejecución
├── src/main/java/ec/edu/uta/sistemaacademico/
│   ├── Estudiante.java   ← la clase (el molde)
│   └── Principal.java    ← método main: crea y muestra los objetos
├── .gitignore            ← archivos que Git no debe subir
├── pom.xml               ← configuración del proyecto Maven
└── README.md             ← documentación del repositorio
```

Clase Estudiante

La clase `Estudiante` (Figura 4) solo declara los cinco atributos, entre las líneas 22 y 34, y cada uno tiene un comentario que explica qué guarda. Al compilar esta clase todavía no existe ningún estudiante: solo se le indica a Java cómo será cada estudiante cuando se cree.

Figura 4

Fragmento de la clase Estudiante.java: declaración de la clase y de sus atributos

```
15  public class Estudiante {
16
17      // ===== Atributos =====
18      // Son las características que comparten todos los estudiantes.
19      // Cada objeto tendrá su propia copia de estos cinco atributos.
20
21      /** Nombre(s) del estudiante. Ejemplo: "Daniela". */
22      public String nombre;
23
24      /** Apellido(s) del estudiante. Ejemplo: "Villacres". */
25      public String apellido;
26
27      /** Edad en años cumplidos. Ejemplo: 18. */
28      public int edad;
29
30      /** Carrera en la que está matriculado. Ejemplo: "Software". */
31      public String carrera;
32
33      /** Semestre (nivel) que cursa actualmente, del 1 al 10. Ejemplo: 1. */
34      public int semestre;
35  }
```

Nota. Se muestran las líneas 15 a 35; el archivo completo se encuentra en el Apéndice A.

Clase Principal

La clase `Principal` contiene el método `main`, que es el punto de entrada del programa. Su código se organizó en tres bloques, identificados con comentarios para que sea fácil de seguir. La Figura 5 muestra los dos primeros.

Figura 5

Fragmento de la clase Principal.java: instanciación y asignación de valores

```
16      public static void main(String[] args) {
17
18          // ===== 1. Instanciación =====
19          // Con "new" se crean tres objetos distintos a partir de la misma clase.
20          Estudiante estudiante1 = new Estudiante();
21          Estudiante estudiante2 = new Estudiante();
22          Estudiante estudiante3 = new Estudiante();
23
24          // ===== 2. Asignación de valores a los atributos =====
25          // Se usa el operador punto (.) para acceder a cada atributo del objeto.
26          estudiante1.nombre = "Daniela";
27          estudiante1.apellido = "Villacres";
28          estudiante1.edad = 18;
29          estudiante1.carrera = "Software";
30          estudiante1.semestre = 1;
31
32          estudiante2.nombre = "Mateo";
33          estudiante2.apellido = "Naranjo";
34          estudiante2.edad = 20;
35          estudiante2.carrera = "Telecomunicaciones";
36          estudiante2.semestre = 3;
37
38          estudiante3.nombre = "Camila";
39          estudiante3.apellido = "Guerrero";
40          estudiante3.edad = 22;
41          estudiante3.carrera = "Medicina";
42          estudiante3.semestre = 6;
```

Nota. Se muestran las líneas 16 a 42; el archivo completo se encuentra en el Apéndice B.

Cada bloque cumple una función concreta dentro del programa:

1. **Instanciación (líneas 18 a 22).** Se crean tres objetos con `new Estudiante()` y cada uno se guarda en su propia variable: `estudiante1`, `estudiante2` y `estudiante3`.
2. **Asignación de valores (líneas 24 a 42).** Con el operador punto se accede a cada atributo del objeto y se le asigna un valor, por ejemplo `estudiante1.nombre = "Daniela";`. Los tres objetos reciben datos diferentes.
3. **Presentación de la información (desde la línea 44).** Con `System.out.println` se muestra cada atributo, uniendo un texto fijo con el valor del objeto mediante el operador `+`.

Ejecución del programa

Para comprobar el funcionamiento compilamos y ejecutamos el programa desde la terminal. La Figura 6 muestra los comandos utilizados y la salida que se obtuvo.

Figura 6

Ejecución del programa en la terminal



Nota. Salida obtenida al compilar con javac y ejecutar con java en OpenJDK 21.

El comando `javac` compila las dos clases y guarda los archivos `.class` en la carpeta `bin`, y el comando `java` ejecuta la clase `Principal`. La información de los tres estudiantes se muestra completa y en el mismo orden en que se asignaron los valores, lo que confirma que cada objeto conserva sus propios datos. El programa también se puede ejecutar con el botón

Run de Visual Studio Code o con *Run Project* en NetBeans; los pasos están detallados en el archivo README.md del repositorio.

Trabajo colaborativo en GitHub

Para trabajar en equipo creamos el repositorio público POO-Tema1-Grupo1, disponible en https://github.com/<usuario>/POO-Tema1-Grupo1. El coordinador del grupo creó el repositorio y agregó a los demás integrantes como colaboradores desde *Settings* > *Collaborators*, lo que permite que cada uno suba cambios con su propia cuenta (GitHub, s.f.). Cada aporte quedó registrado como un *commit*, que funciona como una fotografía del proyecto en un momento dado e indica quién hizo el cambio, cuándo lo hizo y qué modificó (Chacon y Straub, 2014).

Contenido del repositorio

El repositorio contiene el código fuente completo, el archivo `pom.xml`, el archivo `.gitignore` (para no subir archivos compilados ni configuraciones locales), los diagramas en la carpeta `docs`, las capturas en la carpeta `evidencias` y el archivo `README.md`. El README incluye el objetivo, los integrantes, el diagrama de la clase, tanto en imagen como en formato Mermaid que GitHub dibuja automáticamente, la salida esperada y las instrucciones para ejecutar el programa en Visual Studio Code, NetBeans o la terminal.

Distribución de responsabilidades

Repartimos el trabajo de modo que cada integrante tuviera una tarea concreta y al menos un commit verificable. Los commits se hicieron en el orden de la Tabla 3, porque cada parte depende de la anterior: primero la estructura, luego las clases y al final la documentación.

Tabla 3

Distribución de responsabilidades y commits del grupo

N.º	Integrante	Responsabilidad	Archivos	Mensaje del commit
1	Fher Dorian Calderon Carvajal	Coordinación: crea el repositorio, invita a los colaboradores y sube la estructura inicial	`pom.xml`, `.gitignore`	Crea la estructura inicial del proyecto Maven
2	Karina Paola Sánchez Bastidas	Implementa la clase Estudiante con sus cinco atributos	`Estudiante.java`	Agrega la clase Estudiante con sus cinco atributos
3	Brishy Anahy Ashanga Yumbo	Implementa la clase Principal: instancia los tres objetos y muestra sus datos	`Principal.java`	Agrega la clase Principal e instancia tres estudiantes
4	Dereck Yair Acosta Paredes	Elabora los diagramas de clase y de objetos	`docs/`	Agrega los diagramas de clase y de objetos
5	Antoni Casillas	Redacta el README y agrega las evidencias de ejecución	`README.md`, `evidencias/`	Documenta el proyecto y agrega evidencias de ejecución

Nota. Cada integrante realizó su commit desde su propia cuenta de GitHub, como colaborador del repositorio.

Evidencias de las contribuciones

La Figura 7 muestra la página principal del repositorio y la Figura 8 el historial de commits, donde se puede verificar el aporte individual de cada integrante.

Figura 7

Página principal del repositorio POO-Tema1-Grupo1 en GitHub

Pegar aquí la captura de la página principal del repositorio (lista de archivos con el README visible).

Figura 8

Historial de commits con las contribuciones de los integrantes

Pegar aquí la captura de la sección Commits o de Insights > Contributors, donde se vea el nombre de cada integrante junto a su commit.

Reflexión final

¿Por qué una clase puede utilizarse para crear varios objetos?

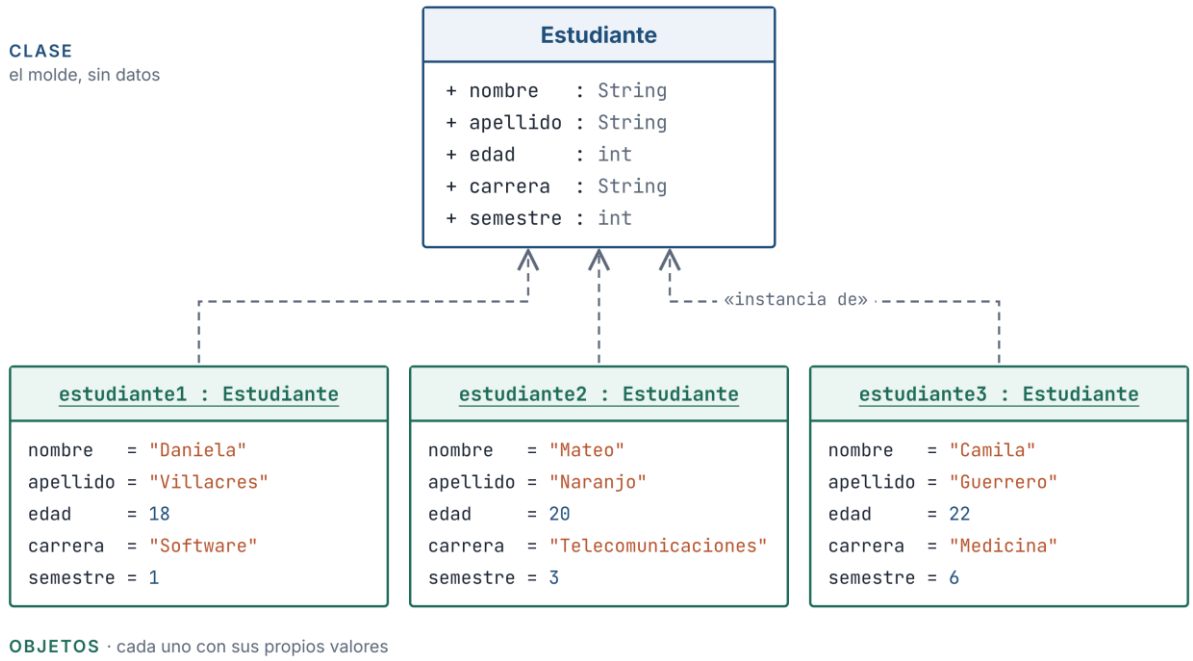
Porque la clase no guarda datos, solo describe la estructura que tendrán los objetos. Cada vez que se usa `new`, Java reserva un espacio nuevo en la memoria con una copia propia de todos los atributos, así que crear un objeto no afecta a los que ya existen. En nuestro programa, la misma clase `Estudiante` sirvió para crear a Daniela, a Mateo y a Camila sin tener que escribir tres clases distintas. Es lo mismo que ocurre con la plantilla del carnet: se diseña una vez y se imprime todas las veces que haga falta.

¿Qué diferencias existen entre los atributos definidos en la clase y los valores de cada objeto?

La Figura 9 representa los tres objetos que se crean en la clase `Principal`. Los nombres de los atributos son los mismos en todos, pero los valores cambian de un objeto a otro.

Figura 9

Diagrama de los objetos creados en la clase Principal



Nota. En UML, el nombre de un objeto se subraya y se escribe junto a su clase. Las flechas punteadas indican que cada objeto es una instancia de Estudiante.

Los atributos se definen una sola vez en la clase y son iguales para todos los objetos: todos tienen nombre, apellido, edad, carrera y semestre, con el mismo tipo de dato. Los valores, en cambio, pertenecen a cada objeto y pueden ser diferentes. Por ejemplo, `estudiante1.edad` vale 18 y `estudiante3.edad` vale 22, y si se cambia la edad de uno, la del otro no se modifica. La clase indica qué se guarda; cada objeto guarda su propio dato.

¿Cómo podría aprovecharse la clase Estudiante dentro del proyecto integrador de la asignatura?

Nuestro proyecto integrador es VitalCore Web, una aplicación web para la gestión de historias clínicas. Aunque no es un sistema académico, la clase `Estudiante` nos sirve como modelo para las clases que vamos a necesitar. Un paciente, por ejemplo, también tiene nombre, apellido y edad, así que la clase `Paciente` se puede construir con la misma lógica y

agregarle atributos propios, como la cédula o el tipo de sangre. Lo mismo pasa con clases como `Medico` o `Cita`. Además, si el sistema llegara a registrar a estudiantes de medicina o a internos que realizan prácticas, la clase `Estudiante` se podría reutilizar casi sin cambios.

Más allá del código, lo que más nos sirve para el proyecto es el proceso que seguimos: identificar las entidades del problema, decidir qué atributos necesita cada una, dibujarlas en UML y recién después programarlas.

Conclusiones

La práctica nos permitió pasar de la definición teórica de clase y objeto a un programa que funciona. Comprobamos que una sola clase, `Estudiante`, basta para crear varios objetos con datos diferentes, y que agrupar los datos en objetos hace que el código sea mucho más ordenado que trabajar con variables sueltas. También aprendimos a representar la clase en UML antes de programarla, lo que hizo que la implementación fuera más rápida y con menos errores.

El trabajo en GitHub nos obligó a organizarnos: cada integrante tuvo una tarea concreta y su aporte quedó registrado en el historial de commits. Como siguiente paso, la clase `Estudiante` puede mejorarse con conceptos que todavía no se aplicaron en este tema, como los constructores, los métodos y el encapsulamiento.

Referencias

Booch, G., Maksimchuk, R. A., Engle, M. W., Young, B. J., Conallen, J. y Houston, K. A. (2007). *Object-oriented analysis and design with applications* (3.ª ed.). Addison-Wesley.

Chacon, S. y Straub, B. (2014). *Pro Git* (2.ª ed.). Apress. https://git-scm.com/book/es/v2

Deitel, P. y Deitel, H. (2016). *Cómo programar en Java* (10.ª ed.). Pearson Educación.

Fowler, M. (2004). *UML distilled: A brief guide to the standard object modeling language* (3.ª ed.). Addison-Wesley.

GitHub. (s.f.). *Invitar colaboradores a un repositorio personal*. GitHub Docs. Recuperado el 15 de octubre de 2026, de https://docs.github.com/es/repositories/managing-your-repositorys-settings-and-features/repository-access-and-collaboration/inviting-collaborators-to-a-personal-repository

Joyanes Aguilar, L. y Zahonero Martínez, I. (2011). *Programación en Java 6: Algoritmos, programación orientada a objetos y UML*. McGraw-Hill.

Oracle. (s.f.-a). *Lesson: Object-oriented programming concepts*. The Java Tutorials. Recuperado el 15 de octubre de 2026, de https://docs.oracle.com/javase/tutorial/java/concepts/

Oracle. (s.f.-b). *Creating objects*. The Java Tutorials. Recuperado el 15 de octubre de 2026, de https://docs.oracle.com/javase/tutorial/java/javaOO/objectcreation.html

Apéndice A

Código completo de la clase Estudiante.java

```java
package ec.edu.uta.sistemaacademico;

/**
 * Representa a un estudiante dentro del sistema académico de la
 * Universidad Técnica de Ambato.
 *
 * <p>Esta clase es el "molde" del sistema: define qué datos tiene todo
 * estudiante, pero no guarda los datos de ninguno en particular. Los
 * datos concretos se guardan en cada objeto que se crea a partir de
 * ella con la palabra reservada {@code new}.</p>
 *
 * @author Grupo 1 - POO Tema 1
 * @version 1.0
 */
public class Estudiante {

    // ===== Atributos =====
    // Son las características que comparten todos los estudiantes.
    // Cada objeto tendrá su propia copia de estos cinco atributos.

    /** Nombre(s) del estudiante. Ejemplo: "Daniela". */
    public String nombre;

    /** Apellido(s) del estudiante. Ejemplo: "Villacres". */
    public String apellido;

    /** Edad en años cumplidos. Ejemplo: 18. */
    public int edad;

    /** Carrera en la que está matriculado. Ejemplo: "Software". */
    public String carrera;

    /** Semestre (nivel) que cursa actualmente, del 1 al 10. Ejemplo: 1. */
    public int semestre;
}
```

Apéndice B

Código completo de la clase Principal.java

```java
package ec.edu.uta.sistemaacademico;

/**
 * Clase principal del programa.
 *
 * <p>Contiene el método {@code main}, que es el punto de entrada de la
 * aplicación. Aquí se crean (instancian) tres objetos de tipo
 * {@link Estudiante}, se asignan valores diferentes a sus atributos y se
 * muestra la información de cada uno en la consola.</p>
 *
 * @author Grupo 1 - POO Tema 1
 * @version 1.0
 */
public class Principal {

    public static void main(String[] args) {

        // ===== 1. Instanciación =====
        // Con "new" se crean tres objetos distintos a partir de la misma clase.
        Estudiante estudiante1 = new Estudiante();
        Estudiante estudiante2 = new Estudiante();
        Estudiante estudiante3 = new Estudiante();

        // ===== 2. Asignación de valores a los atributos =====
        // Se usa el operador punto (.) para acceder a cada atributo del objeto.
        estudiante1.nombre = "Daniela";
        estudiante1.apellido = "Villacres";
        estudiante1.edad = 18;
        estudiante1.carrera = "Software";
        estudiante1.semestre = 1;

        estudiante2.nombre = "Mateo";
        estudiante2.apellido = "Naranjo";
        estudiante2.edad = 20;
        estudiante2.carrera = "Telecomunicaciones";
        estudiante2.semestre = 3;

        estudiante3.nombre = "Camila";
        estudiante3.apellido = "Guerrero";
        estudiante3.edad = 22;
        estudiante3.carrera = "Medicina";
        estudiante3.semestre = 6;

        // ===== 3. Presentación de la información =====
        System.out.println("================================================");
        System.out.println("        SISTEMA DE REGISTRO DE ESTUDIANTES");
        System.out.println("      Universidad Tecnica de Ambato  -  UTA");
        System.out.println("================================================");

        System.out.println();
        System.out.println("[ Estudiante 1 ]");
        System.out.println("  Nombre   : " + estudiante1.nombre);
        System.out.println("  Apellido : " + estudiante1.apellido);
        System.out.println("  Edad     : " + estudiante1.edad);
        System.out.println("  Carrera  : " + estudiante1.carrera);
        System.out.println("  Semestre : " + estudiante1.semestre);

        System.out.println();
        System.out.println("[ Estudiante 2 ]");
        System.out.println("  Nombre   : " + estudiante2.nombre);
        System.out.println("  Apellido : " + estudiante2.apellido);
        System.out.println("  Edad     : " + estudiante2.edad);
        System.out.println("  Carrera  : " + estudiante2.carrera);
```

```
64          System.out.println("  Semestre : " + estudiante2.semestre);
65
66          System.out.println();
67          System.out.println("[ Estudiante 3 ]");
68          System.out.println("  Nombre   : " + estudiante3.nombre);
69          System.out.println("  Apellido : " + estudiante3.apellido);
70          System.out.println("  Edad     : " + estudiante3.edad);
71          System.out.println("  Carrera  : " + estudiante3.carrera);
72          System.out.println("  Semestre : " + estudiante3.semestre);
73
74          System.out.println();
75          System.out.println("================================================");
76          System.out.println("  Total de estudiantes registrados: 3");
77          System.out.println("================================================");
78      }
79  }
```